The FLOPs You Save Are Not the Time You Spend: Sparse Attention in Multi-Agent Communication

Anonymous authors
Paper under double-blind review

Abstract

Attention-based communication between agents is routinely sparsified on the grounds that attending to a few peers rather than to all of them removes most of the work, and the case is almost always made by counting operations rather than by measuring execution. We show that the count is smaller than it appears, and that what it buys in time is smaller still. When an agent's partners are chosen by their attention scores, every score has already been computed before any can be discarded, so only the aggregation is removable and no choice of sparsity can remove more than half the arithmetic of attention. Whether even that half becomes time depends on the implementation to a degree an operation count cannot see. Applying the selection as a mask leaves the dense aggregation in place and executes no saving at all; a genuinely sparse implementation assembled from general-purpose tensor operations moves several times more data than dense attention despite performing less arithmetic, and runs an order of magnitude slower. Writing a fused kernel that keeps every intermediate on chip removes most of that penalty, and at the very sparsest settings it makes attention over selected peers faster than the best dense kernel we could write, by a margin within the analytic ceiling and confined to retaining one or two peers. At the sparsity levels multi-agent communication actually uses, the fastest sparse implementation we could construct is still far slower, because ranking the candidates costs more than aggregating over the survivors saves. Genuine savings therefore have to come from deciding what to attend to before the scores exist, and from executing that decision in a kernel written for it; we measure two such selectors, and the difference between them shows that escaping the operation count is necessary but not sufficient.

1 Introduction

Cooperative multi-agent systems increasingly pass learned messages between agents, and attention is the mechanism they use to decide whose message matters (Das et al., 2019; Jiang & Lu, 2018; Jiang et al., 2020). It is a natural choice: the weighting is learned, it adapts to the situation, and it needs no hand-designed communication topology. It is also pairwise, so the channel costs quadratically in the number of agents, and a large part of the literature on learned communication is concerned with restricting who talks to whom.

Restricting communication can mean two different things, and the difference is easy to lose. In a distributed deployment the objective is to reduce the messages physically transmitted, and any restriction achieves that in proportion to the links it removes. In the centralised inference that most of this literature actually runs, the objective is to reduce the computation performed on one device, and there the proportionality does not follow. This paper is about the second. It asks what a particular and natural restriction — keeping, for each agent, the peers with the highest attention scores — saves in local computation, and whether the saving appears as elapsed time.

The efficiency case for that restriction is easy to state and easy to accept. If dense aggregation combines messages from every peer and sparse aggregation combines only a handful, then almost all of the aggregation disappears. What the statement omits is where the handful comes from. Selecting peers by attention score

requires the scores, so by the time the selection can be made, every pairwise interaction has been evaluated. Attention has two matrix products of equal size, one producing the scores and one aggregating the values; sparsity of this kind reaches only the second. The arithmetic that can be removed is therefore at most half, whatever the sparsity, and the familiar fraction that describes the aggregation term alone overstates the achievable saving by a factor approaching two.

An operation count is a model of cost, and it can fail in more than one way. It can describe a computation the implementation does not perform: applying the selection as a mask on the dense attention matrix materialises the discarded entries as zeros and multiplies them anyway, so the executed aggregation is unchanged while a count based on the retained fraction reports a large saving. And the quantity it describes may not govern the outcome: on hardware where time tracks bytes moved rather than arithmetic performed, removing arithmetic from a kernel that is already waiting on memory changes nothing. Both failures occur here, and the second is the one that survives every correction we could make to the first.

The question this paper answers empirically is therefore whether genuinely executing the reduced aggregation makes score-then-select communication faster in the agent-count regime multi-agent communication occupies. We answer it by measuring six implementations of one function, verified equivalent at every configuration whose latency we report, against dense attention through both a library kernel and one we wrote ourselves.

The answer is conditional, and the condition is the contribution. Assembled from general-purpose tensor operations, the sparse path is slower than dense attention everywhere we measure, by up to an order of magnitude, and it moves several times more memory traffic while performing less arithmetic. A fused kernel that keeps the score row in registers and writes no intermediate removes most of that penalty; at the very sparsest settings it makes the sparse path genuinely faster than the best dense implementation available to us, by a margin that stays inside the analytic ceiling. The advantage does not survive an increase in the number of retained peers, because ranking candidates costs more than aggregating over survivors saves, and it is gone well before the sparsity levels this literature uses. A kernel can make selection cheap in the number retained only by abandoning the compaction that made the aggregation sparse in the first place, so on this hardware a score-then-select kernel can have sparse arithmetic or cheap selection but not both.

What follows constructively is the mechanism the negative result identifies. Selection that fixes the retained set before any score exists is not subject to the ceiling, and its operation-count saving is unbounded in principle. Whether it converts into time is a separate question, and we answer it for two selectors the count treats similarly: choosing the nearest peers by distance, which is what neighbourhood-based communication already does, and ordering agents along a space-filling curve, which is asymptotically cheaper. They behave oppositely as the agent count grows. The first saves an order of magnitude of arithmetic and loses an order of magnitude of time, because it ranks a pairwise matrix of its own; the second makes attention over the retained peers genuinely faster than dense attention, and is held above parity only by the fixed cost of its own sort. Escaping the operation-count bound is necessary, is not sufficient, and does not by itself say which of two escaping selectors will be faster.

Contributions.

- We show that any rule selecting communication partners by their attention scores can remove at most half of attention's arithmetic, however sparse it is, and that the bound is the special case of a general accounting in which a selection rule is charged for what it costs to run. That accounting places existing multi-agent restriction mechanisms on one axis and separates those that escape the bound from those that inherit it (Section 3).

- We measure what is actually executed, distinguishing masked, gathered and fused implementations of the same function, and find that the composed sparse path is slower than dense attention throughout the regime by a margin its operation count cannot predict, with memory traffic rather than arithmetic as the cause (Section 4).

- We write fused sparse kernels and a matched fused dense baseline, and thereby separate what the algorithm costs from what the implementation costs. Sparsity does become faster than dense

attention, but only when one or two peers are retained, by less than the analytic ceiling permits, and never at the sparsity multi-agent communication uses (Section 4.3).

- We instantiate the constructive half of the analysis with two selectors that fix the retained set before scoring, and show that removing the operation-count bound is necessary but not sufficient. A selector can save the arithmetic the accounting promises and cost an order of magnitude in time if it reaches its decision through a structure of the size it meant to avoid; and a selector that avoids that can make the attention itself faster than dense and still lose on its own fixed cost, which at these agent counts is comparable to the attention it displaces (Section 5).

A secondary pre-registered training study establishes that the architecture being sparsified improves return, so that the sparsification comparisons are not vacuous, and reports six under-powered null contrasts as what they are (Section 6). Every computational claim concerns centralised inference on a single device, not bandwidth-constrained distributed deployment; Section 8 states the full envelope.

2 Related Work

2.1 Communication sparsity in multi-agent reinforcement learning

Learned inter-agent communication has converged on attention as the mechanism for deciding whose message matters. CommNet (Sukhbaatar et al., 2016) broadcasts mean-pooled hidden states; TarMAC (Das et al., 2019) introduces signature-based targeting so that receivers weight senders; MAT (Wen et al., 2022) casts the joint policy as a sequence model. Applied systems inherit the pattern, pairing heterogeneous learners with attention-based message aggregators (Hamdan et al., 2026). Since attention over N agents is pairwise, a large part of this literature also restricts who talks to whom, and the restriction is usually motivated by cost. DGN observes that receiving a large amount of information "requires high bandwidth and incurs high computational complexity" (Jiang et al., 2020), a claim about transmitted bytes and about local compute, of which only the second is what we measure.

Proposition 2 organises these restriction mechanisms by what their selector costs, and the resulting three groups behave differently. The first fixes the retained set from structure that is available before any score exists: DGN restricts attention to each agent's k nearest neighbours by Euclidean distance (Jiang et al., 2020), ATOC's initiator forms a group from its nearest collaborators within a communication field (Jiang & Lu, 2018), and mean-field methods replace the peer set with a single averaged interaction (Yang et al., 2018). Their selectors are free or nearly so, which places them in case (i) of the proposition. The second group scores something cheaper than the full pairwise interaction: IC3Net gates whether an agent speaks at all (Singh et al., 2019), and SchedNet ranks agents by a scalar importance weight and schedules the k largest (Kim et al., 2019). These are top-k rules, but over an $O(N)$ vector rather than an $N \times N$ matrix, so Proposition 1 does not bind them. The third group runs a pairwise selector before running pairwise attention: G2ANet decides which of all N^2 edges survive with a hard-attention network and then applies soft attention to the survivors (Liu et al., 2020b), and MAGIC builds a hard or soft adjacency from an all-pairs interaction before its graph attention (Niu et al., 2021). Their arithmetic saving is real in the aggregation and paid for in the selector, at the same order as the score matrix they avoid.

The mechanism this paper analyses — selecting peers by the attention score itself, after computing all of them — is the limiting case of that third group, and it is rare in multi-agent communication rather than absent. The closest published relative is the adaptive sparse communication graph of Sun et al. (2020), which replaces softmax with a generalisation of sparsemax so that the normalisation itself emits exact zeros. It motivates the design partly through scalability, stating that the resulting sparsity "can reduce the complexity of both the forward pass and the back-propagation of the policy and value networks", and it explicitly considers and rejects hard top-k thresholding as non-differentiable. Because the sparsity-inducing map is applied to logits already formed for every pair, its selection is post-score and sits inside the class Proposition 1 constrains. That is a statement about where the method lands in our accounting rather than a contradiction of its claims: its reported gains are in sample efficiency at up to 30 agents and it reports no measured runtime.

Handshake-style mechanisms that match a requester's query against every candidate key and then select (Liu et al., 2020a) are score-then-select in the same sense.

Two consequences for scope. The design we analyse is one we proposed and built, so the ceiling constrains our own design and the structural selectors above are outside it. And the agent counts at which these methods operate are small: between two and roughly thirty agents for the attention-based communication methods above, with DGN reaching the low hundreds in grid worlds. Our sweep runs to N=512 so that the trend is visible, but the regime the argument is about is $N \lesssim 10^2$, and beyond it the field has already abandoned all-pairs interaction in favour of neighbourhoods and mean fields — which is to say, in favour of Proposition 2.

2.2 Sparse and efficient attention

The efficient-attention literature is where the operation-count argument for sparsity is most developed, and its mechanisms separate along the same axis. Longformer (Beltagy et al., 2020) and BigBird (Zaheer et al., 2020) fix which pairs interact in advance through sliding windows, global tokens and random edges, so the score computation itself becomes sparse and the saving is asymptotic; their selector is essentially free. Reformer (Kitaev et al., 2020) restricts candidate interactions by a locality-sensitive hash and Routing Transformer (Roy et al., 2021) by online k-means clustering; both are content-dependent, both evaluate scores only for candidate pairs, and together they are why the escape proposition carries a selector term at all. Performer (Choromanski et al., 2021) falls outside the premise entirely: it keeps every interaction and reorders the algebra through a kernel factorisation.

Top-k attention in the sequence setting is the closest external analogue of the mechanism we build, and the comparison turns on which efficiency is being claimed. Gupta et al. (2021) process queries in chunks and, for each query, compute its k largest scores against all keys exactly — a score-then-select rule in our sense — and their stated benefit is memory: chunking reduces peak attention memory from $\Omega(L_Q L_K)$ to $O(C L_K)$ for chunk size C while remaining a drop-in replacement in pretrained models. Memory efficiency, operation-count efficiency and wall-clock efficiency are distinct quantities, and a mechanism can improve one at the expense of another; ours is a question about the second and the third.

A related body of work asks how much of the attention function survives truncation, bounding the approximation error of keeping only the largest entries. That question is orthogonal to ours. We hold the function fixed — our gathered, masked and fused top-k implementations agree to floating-point tolerance — and ask whether the sparsification reduces executed computation and elapsed time. A method could be exact and slow, or approximate and fast, and the two literatures answer different halves of the design question.

Sparsemax (Martins & Astudillo, 2016) and α-entmax (Peters et al., 2019) obtain exact zeros from the normalisation rather than a hard cutoff, and make no claim about attention cost; that their zeros arrive after the scores exist, and are therefore subject to our ceiling, is an inference we draw rather than a claim they make. Gumbel-softmax (Jang et al., 2017) supplies the relaxation used by learned discrete gates, and constrained-RL formulations (Achiam et al., 2017; Stooke et al., 2020) the machinery for optimising under a communication budget. We implement entmax and a Lagrangian budget as arms in Section 6.

Score-then-select is used at scale in sparse mixture-of-experts, where a router scores every expert and dispatches each token to the best k (Shazeer et al., 2017; Fedus et al., 2022). The structure of the argument is the one we examine, and so is the exemption: these architectures are designed so that the router is small relative to the expert computation it gates, so the analogue of our $N^2 d$ score term is a minor cost rather than an equal one, and the saving is real. Whether score-then-select pays therefore depends on the ratio between the cost of scoring all candidates and the cost of the work being skipped — large in mixture-of-experts, and equal to one in single-head attention, which is Proposition 1.

2.3 Hardware-aware sparse execution

That operation counts, memory traffic, kernel fusion and realised latency are distinct quantities is established in the systems literature. The roofline model (Williams et al., 2009) formalises when arithmetic ceases to govern runtime. FlashAttention (Dao et al., 2022) and memory-efficient attention (Rabe & Staats, 2021)

make the point from the other direction: the winning implementations of dense attention reduce memory traffic without changing the operation count at all, which is precisely the axis a FLOPs-based argument cannot see. Longformer anticipates the distinction this paper turns on, reporting runtime and memory for three implementations of one attention pattern — a loop version, a vectorised chunked version and a custom CUDA kernel — and noting that the chunked variant consumes roughly twice the optimal memory because it computes some zero values.

Recent systems work establishes where sparse attention does pay, and it bounds our conclusion from the other side. Native Sparse Attention (Yuan et al., 2025) ranks compressed key–value blocks and attends within only the selected blocks, with kernels written for the resulting access pattern, reporting large forward, backward and decoding speedups at 64k context; MiniMax Sparse Attention (Lai et al., 2026) co-designs blockwise selection with its GPU execution path and reports comparable prefill and decoding gains at 1M context. Three properties separate that regime from ours, and all three are load-bearing. Selection is amortised over blocks rather than performed per pair, so the selector cost falls by the block size. Block structure preserves the dense matrix-multiply shape inside each selected tile, so the arithmetic that remains still runs on the hardware path built for it — which per-pair gather, whose aggregation degenerates into a matrix–vector product, does not. And at 10^5–10^6 tokens the skipped computation is enormous, where at $N \lesssim 10^2$ it is bounded by Proposition 1 at a factor of two. We take these results at face value and make no claim of comparability across four orders of magnitude in sequence length; what they identify is what a successor design in our regime would have to change.

Measurement practice. Our training protocol follows the recommendations of Papoudakis et al. (2021) for systematic MARL benchmarking, and we report effect sizes and intervals rather than bare means (Agarwal et al., 2021). Environments come from the PettingZoo/MPE family (Terry et al., 2021) and the learner is MAPPO (Yu et al., 2022). Motivated by Longformer's precedent, we also checked whether the efficiency claims in the papers we drew on are accompanied by a measurement: among the multi-agent papers making such a claim, one of five reports a measured temporal quantity attributable to its mechanism, against five of six among the efficient-attention and mixture-of-experts papers. Twelve papers support no conclusion about either field, and we report the asymmetry only because it characterises the evidence base from which we ourselves adopted the operation-count argument. Appendix D gives the protocol, the completed sheet and its scope caveats.

3 Computational Limits of Score-Then-Select Attention

3.1 Problem setting

Consider one round of scaled dot-product attention over N agents, per batch element, at head width d. Agent i forms a query q_i; every agent contributes a key and a value; self-attention is masked. Write $s_i = q_i K^\top \in \mathbb{R}^N$ for the score vector and $a_i = \mathrm{softmax}(s_i)$ for the attention weights, so that dense attention computes $m_i = a_i V$. A top-k variant keeps the k largest entries of a_i and renormalises over them.

The mechanism this paper analyses is the one that arises most naturally from that description: form the complete score vector, then keep its k largest entries. We call such a rule *score-then-select*, and the term is used in its exact sense throughout. Every analytic claim below concerns that class, not content-dependent selection in general, and Section 3.2 draws the boundary.

Attention has two matrix products of equal size. One produces the pairwise scores; the other aggregates the values. Sparsifying the retained set shrinks the second, because only k value rows are combined. It cannot shrink the first, because the scores are what the selection reads. That observation is the whole of Section 3.2; the rest of this section makes it precise and marks where it stops applying.

Top-k attention admits three implementations of the identical function, and the distinction between them is what separates an arithmetic saving from an executed one.

Masked top-k. Compute a_i in full, multiply by a $\{0,1\}$ mask, renormalise, and evaluate $m_i = \tilde{a}_i V$ with $\tilde{a}_i$ still dense in memory. The discarded entries are materialised as zeros and multiplied, so the



Every implementation evaluates every pairwise score, because a score-then-select rule must form the complete score row before it can rank it. A dashed outline marks weights that are used but never written to memory; a cross marks the masked self-interaction.

Figure 1: Two implementations of dense attention and three of top-k attention, computing two functions in five ways. All five evaluate every pairwise score, because a score-then-select rule must form the complete score vector before it can rank it. They differ in what reaches memory and in how much arithmetic the aggregation performs, and only the gathered and fused top-k variants execute the saving that motivates sparsification at all.

executed aggregation count is unchanged from dense while selection, masking and renormalisation are added on top.

Gathered top-k. Take the top-k of the raw scores, softmax over those k values, gather the corresponding value rows into a (B, N, k, d) tensor, and contract $(B{\cdot}N, 1, k) \times (B{\cdot}N, k, d)$. No zeros are materialised and the aggregation genuinely shrinks.

Fused top-k. Compute the same function in a single kernel, holding the score row in registers, selecting within them, and accumulating the weighted values without writing a score matrix, an index tensor or a gathered value tensor to memory. Section 4.3 reports one.

Gathered and masked top-k compute the same function (Figure 1). Softmax is monotone within a row, so the top-k weights are the top-k scores, and a softmax over just those k scores equals the dense softmax restricted to them and renormalised; the two agree to $< 10^{-10}$ in `float64` (Appendix B). The operation-count argument is about the gathered variant. Timing the masked variant measures an implementation artifact, and timing either against unfused dense alone ignores what a production kernel costs, so we report all implementations against both dense baselines throughout.

3.2 An operation-count ceiling

Analytic counts are multiply–accumulates. Under the convention that a multiply–add is two floating-point operations every term doubles and every ratio is unchanged, so the ceiling is the same number either way; we write MACs where the count is analytic and FLOPs only where a counter attached to the running program

produced the figure. Counting the two matrix products that constitute attention, and excluding the shared input and output projections:

$$C_{\text{scores}} = N^2 d, \quad C_{\text{agg}} = N^2 d, \quad C_{\text{dense}} = 2N^2 d, \qquad C_{\text{top-}k} = \underbrace{N^2 d}_{\text{all scores}} + \underbrace{Nkd}_{\text{aggregate } k}. \tag{1}$$

Proposition 1 (Score-then-select ceiling)**.** *Let the retained set be fixed by a score-then-select rule, one whose kept indices are determined only after the complete score vector of each query has been formed. Then*

$$\frac{C_{dense}}{C_{top\text{-}k}} = \frac{2N^2 d}{N^2 d + Nkd} = \frac{2N}{N+k} < 2 \qquad \textit{for all } k \geq 1, \tag{2}$$

with supremum 2 *approached only as* $k/N \to 0$*. Within this class, no choice of* k *or* N *yields more than a twofold reduction in attention MACs.*

Proof. $C_{\text{top-}k} > N^2 d = \frac{1}{2} C_{\text{dense}}$ for every $k \geq 1$ since $Nkd > 0$, so the ratio is below 2. It is decreasing in k, hence largest at $k = 1$, and approaches 2 from below as $k/N \to 0$. □

The familiar figure $1 - k/(N-1)$ describes the value-aggregation term alone. Read as *the* saving it overstates the achievable reduction by a factor approaching two, before any question of whether the reduction is realised in time.

We use $N^2 d$ rather than $N(N-1)d$ because it is the count the implementations execute: self-masking writes $-\infty$ into the diagonal of a full $N \times N$ score matrix rather than skipping work. Counting the diagonal exactly gives $2(N-1)/\big((N-1)+k\big)$, which is strictly smaller for every $1 \leq k \leq N-1$, so the proposition is a valid upper bound under either convention.

Five assumptions support the statement, of which only the second is essential. **(A1) Single round**: with R rounds both terms scale by R and the ratio is unchanged. **(A2) Full-score selection**: the retained set is determined only after the complete candidate score vector has been computed, so all N scores must be formed. **(A3) Per head**: with H heads of width d/H both terms scale identically. **(A4) Projections excluded**: including the shared qkv and output projections moves the ratio strictly closer to 1, so the bound remains valid. **(A5) Selection cost excluded**: counting the top-k operation itself can only decrease the ratio, and Section 4.5 shows the term is large in practice.

A2 concerns *when* the retained set is fixed, not whether the rule depends on the data. Mechanisms that are content-dependent but not score-then-select lie outside the class and are unconstrained by Proposition 1: exact or approximate nearest-neighbour retrieval, clustering and routing, indexing and hashing can all identify candidates without materialising the complete pairwise score matrix. Routing Transformer's online k-means routing (Roy et al., 2021) and Reformer's locality-sensitive hash (Kitaev et al., 2020) are content-dependent in every ordinary sense and still evaluate scores only for candidate pairs; a hierarchical rule that ranks compressed blocks and then scores only the selected ones (Yuan et al., 2025) is likewise outside A2. What such rules pay instead is a selector cost, and the next proposition charges them for it.

3.3 Selection before scoring

Proposition 2 (Escape)**.** *Suppose the kept set is determined* without *forming the complete score vector — structurally, or by a retrieval, routing, clustering, indexing or hashing rule — at total selector cost* $C_{\text{sel}}(N, k, d)$*, and that attention scores are then evaluated only for the* Nk *retained pairs. Then*

$$C_{\text{pre}} = 2Nkd + C_{\text{sel}}, \qquad \frac{C_{dense}}{C_{\text{pre}}} = \frac{2N^2 d}{2Nkd + C_{\text{sel}}}. \tag{3}$$

(i) *If* $C_{\text{sel}} = o(Nkd)$ *— in particular for a fixed or precomputed pattern, where* $C_{\text{sel}} \approx 0$ *— the ratio is asymptotic to* N/k*, that is* $\big(C_{dense}/C_{\text{pre}}\big)/(N/k) \to 1$*. Exact equality with* N/k *holds only when* $C_{\text{sel}} = 0$*; for every* $C_{\text{sel}} > 0$ *the ratio is strictly smaller.* (ii) *If only* $C_{\text{sel}} = o(N^2 d)$*, the ratio is no longer bounded by* 2*: with* $k/N \to 0$ *it diverges, though not in general at the rate* N/k*.*

Proof. Both statements read off the displayed ratio. For (i), $2Nkd + C_{\text{sel}} = 2Nkd\,(1 + o(1))$, so $C_{\text{dense}}/C_{\text{pre}} = (N/k)\,(1 + o(1))^{-1}$, whose quotient with N/k tends to 1; substituting $C_{\text{sel}} = 0$ gives $2N^2d/2Nkd = N/k$ exactly, and any $C_{\text{sel}} > 0$ strictly enlarges the denominator. For (ii), if $k/N \to 0$ then $2Nkd = o(N^2d)$, so the whole denominator is $o(N^2d)$ and the ratio diverges. □

Proposition 1 is the special case $C_{\text{sel}} = N^2d$: a score-then-select rule is one whose selector is a full score matrix, and it is that expense, rather than anything about top-k, that produces the twofold ceiling. Reading the two propositions as one statement places any peer-selection mechanism on a single axis, the cost of deciding what to keep, and the position on that axis determines what the mechanism can save.

The selector-cost term is material rather than decorative. A fixed topology or a precomputed spatial prior really is free at inference and lands in case (i), which is why Longformer and BigBird obtain asymptotic savings where score-then-select top-k cannot (Beltagy et al., 2020; Zaheer et al., 2020): their patterns are fixed before any score is formed, so nothing is spent deciding. A hash must be evaluated and its buckets assembled (Kitaev et al., 2020), and a routing rule must maintain and query its clusters (Roy et al., 2021), so their C_{sel} has to be counted rather than assumed away. Between the two sits a family of multi-agent methods whose selector is itself pairwise — a learned hard-attention or scheduling network evaluated over all N^2 pairs before communication attention runs (Liu et al., 2020b; Niu et al., 2021) — and for these C_{sel} is of the same order as the score matrix they avoid, so they inherit a ceiling of the same kind as Proposition 1 rather than escaping it.

Both propositions concern operation counts only. Whether a pre-score selector attains a proportional *speedup* is a separate question, and one that the measurements in Section 5 answer differently for two selectors that the count treats almost identically.

3.4 Why the count need not govern the time

Proposition 1 bounds a quantity that, in this regime, does not determine runtime. For memory-bound kernels time tracks bytes moved rather than arithmetic performed (Williams et al., 2009), and the composed implementations of Section 3.1 differ far more in bytes than in operations. Gathering does not remove memory traffic; it relocates it, replacing the multiplication of $N - k$ zeros with the reading, writing and re-reading of a compacted (B, N, k, d) tensor and of the indices that build it.

We compute arithmetic intensity, floating-point operations per byte of global memory traffic, analytically per stage, counting each tensor read and written once, and compare it against a machine balance measured on the same device: a large matrix multiply gives the achievable arithmetic rate, a large copy gives achievable bandwidth, and their ratio is the ridge point above which a kernel is compute-bound. Two limits bound the model's use, and both were mis-stated in an earlier version of this work. Counting each tensor once ignores cache residency, so at small N, where the working set fits the GPU's L2, the analytic byte counts overstate traffic; the per-stage accounting of Section 4.5 flags such stages rather than leaving the reader to infer them. And the ridge point depends on whether the probe's buffer fits the last-level cache, so we measure two of them per device. On the GPU the ridge is 23 FLOP/byte cache-resident and 53 from memory; on the host it is 8 and 28. Both paths in this study sit below every one of those figures at every agent count, on both devices, so the memory-bound conclusion does not depend on which ridge is the relevant one — which is a simpler statement than the earlier version could make, because its host bandwidth probe read an unwritten buffer and reported a figure several times too large (Appendix B.5).

Section 4.5 applies this accounting stage by stage and reports, for each stage, how close it runs to whichever machine limit binds it. That is what distinguishes a stage that is badly implemented from one that is doing the only thing its access pattern permits, and the distinction turns out to matter for which parts of the measured penalty a better kernel can remove.

Table 1: Forward latency (ms, minimum over interleaved samples) of one function computed six ways, at a quarter of peers retained. The composed sparse path is slower than every dense implementation at every agent count, and the gap widens with N; the fused sparse kernels change that, and Section 4.3 reports where.

		dense			**top-k**			
N	k	unfused	fused (library)	ours	gathered (composed)	fused gather	fused mask	best sparse ÷ best dense
12	2	0.014	0.013	0.004	0.022	0.004	0.006	1.08
24	4	0.015	0.011	0.004	0.030	0.006	0.018	1.44
48	8	0.069	0.039	0.033	0.073	0.034	0.044	1.03
96	16	0.045	0.031	0.018	0.121	0.049	0.039	2.15
192	32	0.136	0.071	0.037	0.424	0.385	2.218	10.45
384	64	1.082	0.219	0.118	2.405	2.769	2.170	18.35
512	128	1.895	0.347	0.188	6.408	7.221	2.872	15.27

4 Measured Cost

4.1 Method

We time the communication path with the training loop removed. Timing uses on-device CUDA events with an explicit synchronisation on the pinned device, and any diagnostic that transfers a scalar to the host is disabled, because such a transfer synchronises the device inside the timed region. Implementations are timed interleaved repetition by repetition rather than in sequential blocks, so that interference from other tenants lands on all of them at once and their ratios stay paired; the sequential design produced swings above 100% between identical configurations on this shared host. The primary statistic is the minimum over samples, on the modelling assumption that contention adds time rather than removing it, and the whole timed block is repeated so that the spread of per-run medians is reported rather than hidden (Appendix A.2). GPU clocks could not be locked; the account lacks the privilege. Executed operation counts come from `torch.utils.flop_counter`, so latency is always reported beside the arithmetic actually performed.

Two decisions about what is compared deserve stating, because both changed our conclusions. First, we time the attention core — from queries, keys and values to the message tensor — rather than the enclosing module, since the shared input and output projections are identical across implementations and dilute the quantity in question. Second, the dense baseline is the better of PyTorch's fused scaled-dot-product path and a dense attention kernel we wrote ourselves in Triton, with the same tiling policy and in the same session as the sparse kernels. At the head widths multi-agent communication uses, the library path is tuned for a different regime, and comparing a hand-written sparse kernel against it credits sparsity with the difference between two authors (Section 7).

Every implementation is checked against the reference at every configuration whose latency is reported, not at one spot check: exact top-k is discontinuous in the scores, so two implementations that accumulate the score product in different orders can rank a near-tied pair differently and then aggregate over different peers. Maximum deviation across the campaign is at the level of `fp32` rounding (Appendix B).

Unless stated otherwise: PyTorch 2.11 with CUDA 12.8 and Triton 3.6, `fp32`, head width d=16, batch 256, and one NVIDIA Blackwell GPU shared with other tenants.

4.2 What the composed implementations cost

The composed sparse path behaves as the manuscript's earlier version reported. Gathering executes less arithmetic than dense attention — the counter confirms the reduction Proposition 1 permits — and is nonetheless slower than every dense implementation at every agent count we measure (Table 1). Correcting the implementation from masked to gathered helps at small N and stops helping as N grows, because the gather's traffic grows faster than the masked path's wasted arithmetic.

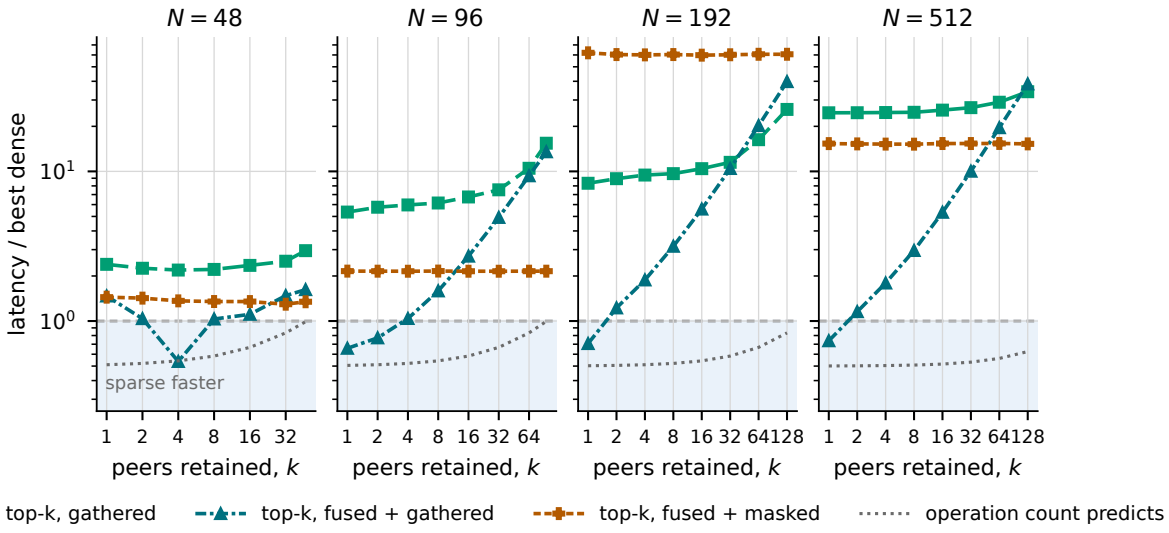


Figure 2: Latency divided by the faster of the two dense implementations, against the number of peers retained. The dotted line is what an operation count predicts for the gathered path, $(N + k)/2N$, which lies below parity everywhere and so predicts a saving at every sparsity. Shading marks the region in which sparsity is actually faster; only the fused gathered kernel enters it, and only at the smallest k.

Table 2: The sparsity at which a purpose-built sparse kernel stops being faster than the best dense implementation, and how far it is from parity at the sparsity multi-agent communication actually uses.

	sparsity that pays		**best ratio**		**at the widest k**	
N	k	of peers	ratio	at k	ratio	at k
96	≤ 2	2.1%	0.66	1	2.2	95
192	≤ 1	0.5%	0.71	1	25.9	128
384	≤ 1	0.3%	0.86	1	18.4	128
512	≤ 1	0.2%	0.74	1	15.3	128

The mechanism is memory traffic rather than arithmetic. Gathering does not remove the score matrix; it adds a compacted (B, N, k, d) tensor and the indices that build it, and then reads both back. Counting bytes gives a criterion that is easier to check than the intensity argument it replaces: gathering is byte-neutral only when the gathered operand is no larger than the score row it avoids reading, that is when $kd \leq N$. At d=16 and N=512 that bound is $k \leq 32$, and the quarter-of-peers operating point is four times past it. The criterion also explains the reversal in which gathering loses to masking at intermediate N, which the earlier version of this manuscript could only attribute to an inference from its byte accounting.

4.3 What a purpose-built kernel changes

The composed path spends most of its time moving intermediates that a fused kernel need not create. To bound how much of the measured penalty is that composition rather than the algorithm, we wrote fused kernels that hold the score row in registers and never write a score matrix, an index tensor or a gathered value tensor to memory. Two selection strategies are available to such a kernel and they trade against each other. Selecting by k successive argmax-and-mask rounds keeps the aggregation genuinely sparse and makes selection cost proportional to k. Selecting by an exact bisection on the integer key of the score makes selection cost independent of k, but expresses the retained set as a zero weight rather than by compaction, so the aggregation performs dense arithmetic however small k is. Both compute the same function as the composed path.

Fusion removes most of the penalty and, at extreme sparsity, reverses the result. At 512 agents retaining a single peer, the composed sparse path costs 4.63 ms and the fused one 0.139 ms, a factor of 33 that is attributable to the implementation and not to the algorithm; the best dense implementation costs 0.188 ms, so the fused sparse path is faster. The manuscript's earlier claim that no sparse implementation available to us is faster than dense attention in any measured configuration does not survive, and we report the correction rather than the claim.

What replaces it is narrower and more useful. A fused sparse kernel is faster than the best dense implementation only when one or two peers are retained — between 0.2% and 2.1% of the available peers, depending on the agent count — and the best ratio anywhere in the sweep is 0.66 (Table 2). At a quarter of peers, which is the setting the multi-agent communication literature uses, the best sparse implementation available to us is 15 to 26 times slower than dense attention.

Three things about that crossover matter more than its existence.

The measured advantage stays inside the analytic ceiling. Proposition 1 permits at most a twofold reduction in operation count, and the largest speedup we measure from sparsity, with implementation held constant, is below that. This is the ceiling behaving as a bound on what is achievable rather than as an abstraction: once the composed implementation's overheads are removed, the residual advantage is the one the count allows and no more. It is also why the first version of these measurements was wrong. Against the library dense path alone the fused sparse kernel appeared more than twice as fast, which exceeds what sparsity can supply and was the signal that the baseline, not the algorithm, was producing the result.

Selection, not aggregation, is what the sparse kernel cannot make cheap. The argmax-rounds kernel performs k row-wide reductions per query, and reductions of this shape run at a small fraction of the arithmetic units' throughput because each is a dependent chain with little instruction-level parallelism. Its cost is accordingly linear in k and dominates as soon as k exceeds a handful. The bisection kernel removes that dependence, and the only way it can is by giving up the compaction — so the implementation whose selection is cheap is the one that performs dense aggregation arithmetic. On this hardware a score-then-select kernel can have sparse arithmetic or k-independent selection, and we could not write one with both.

Fusion is available to dense attention too, and not equally. Dense attention can stream keys and values in blocks with an online softmax, keeping only a small tile resident. Exact score-then-select cannot: the complete score row of a query must exist before the row can be ranked, so the sparse kernel must hold an N-wide tile and its occupancy falls as N grows. That asymmetry is a consequence of assumption A2 rather than of our implementation, and it is why the crossover moves to smaller k as the agent count grows.

4.4 Sensitivity to head width and batch

Extending the sweep to head width 512 resolves a question the earlier version of this manuscript left open, and resolves it against the direction that version extrapolated. Reading the ratio down the head-width axis, the crossover exists only at the two narrowest head widths: the best sparse implementation is faster than the best dense one at head widths 16 and 32 at every agent count, and at no agent count for head width 128 or above, where it is roughly twice as slow (Appendix A). The archival version of this work observed the ratio falling as head width grew and declined to extrapolate; with the implementation corrected, the trend runs the other way.

That is what the byte criterion predicts. Byte-neutrality requires $kd \leq N$, so widening the head at fixed agent count tightens the bound on the retained set proportionally, and at head width 128 the criterion permits only $k \leq 4$ at 512 agents — by which point the selection cost of Section 4.3 has already overtaken the saving. Widening the batch over a thirty-twofold range, by contrast, moves the ratio by less than a fifth and never across parity, which is the expected behaviour of a quantity that is a ratio of per-element costs.

4.5 Where the time goes

Timing each stage in isolation and charging it with its own arithmetic and traffic corrects an explanation the earlier version of this manuscript offered. We had attributed the sparse contraction's slowness to a

Table 3: Per-stage accounting for both paths at N=512. The last column is the fraction of whichever machine limit binds the stage, and it separates a stage that is badly implemented from one doing the only thing its access pattern permits. A figure above 100% marks a stage whose working set is served from cache, so that its analytic byte count is not traffic to memory; those stages are the small ones, and none carries an argument. Peak measured on the same device: 57.8 TFLOP/s and 1497 GB/s, a ridge point of 39 FLOP/byte.

stage	GFLOP	GB	FLOP/byte	ms	TFLOP/s	GB/s	% of limit
score product	2.15	0.29	7.53	0.549	3.91	520	35
softmax over N	0.34	0.54	0.62	0.336	1.00	1598	107
dense aggregation	2.15	0.29	7.53	0.235	9.13	1213	81
top-k selection	0.00	0.47	—	3.271	—	143	10
softmax over k	0.08	0.13	0.62	0.059	1.42	2268	151
gather of V	0.00	1.21	—	0.954	—	1265	84
sparse aggregation	0.53	1.14	0.47	0.816	0.65	1398	93

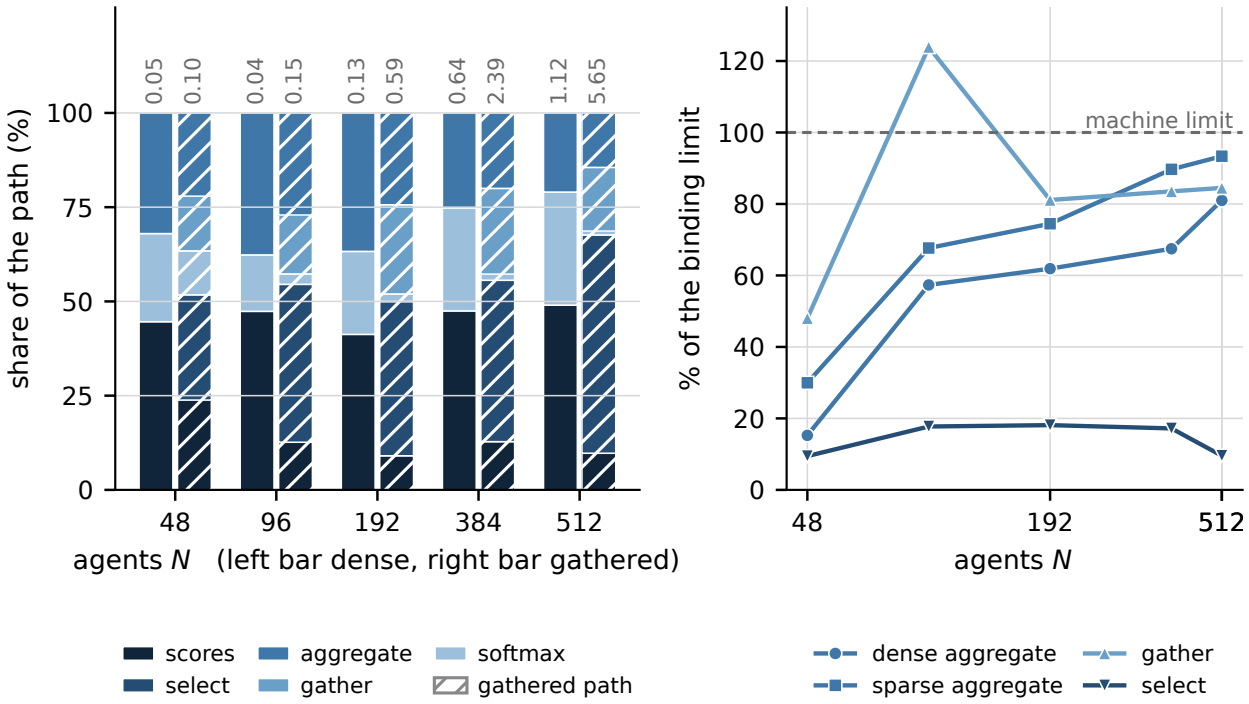


Figure 3: **Left:** how each path divides its time. Selection grows to dominate the gathered path as the agent count rises. **Right:** how close each stage runs to the machine limit that binds it.

batched matrix–vector product having lower arithmetic intensity than a real matrix multiply and being unable to reach the same hardware paths. Measured, that is wrong. The sparse contraction reaches 93% of the bandwidth the device attains on a large copy, against 91% for the dense multiply it replaces (Table 3). Both are bandwidth-saturated and neither is on a worse hardware path; the sparse one is slower because it moves about four times the bytes, since the compacted operand carries kd elements per query where the score row carries N. The corrected account is simpler, checkable before the code is written, and it is the $kd \leq N$ criterion of Section 4.2.

Selection is a different case, and it is the one stage where the raw decomposition is misleading in the opposite direction. It is the largest stage of the composed sparse path, and it runs at under a tenth of the bandwidth the device attains: it must read the score matrix and write values and indices, and moving those bytes and nothing else would take a small fraction of the time it does. That headroom is real, and the fused kernels

Table 4: Forward, backward and their sum, for the differentiable implementations. The sparse path's backward is cheap relative to its forward, because selection has almost no adjoint, so the gap against fused dense narrows without closing.

		dense, unfused			dense, fused			top-k masked			top-k gathered			
N	k	f	b	f+b	f	b	f+b	f	b	f+b	f	b	f+b	gathered f+b ÷ fused f+b
48	8	0.069	0.172	0.241	0.039	0.166	0.205	0.093	0.277	0.370	0.073	0.222	0.294	1.4
96	16	0.045	0.058	0.102	0.031	0.106	0.138	0.129	0.111	0.240	0.121	0.067	0.188	1.4
192	32	0.136	0.171	0.307	0.071	0.219	0.290	0.454	0.462	0.916	0.424	0.201	0.625	2.2
384	64	1.082	1.490	2.572	0.219	0.727	0.945	3.069	3.245	6.313	2.405	1.500	3.906	4.1
512	128	1.895	3.044	4.940	0.347	1.203	1.550	6.959	7.691	14.650	6.408	3.425	9.832	6.3

Table 5: Where one MAPPO iteration spends its time. Differences between communication implementations are at or below the iteration-to-iteration spread in the rightmost column.

		share of the iteration (%)							
N	communication	environment	acting	update fwd	update bwd	optimiser	s/iter	÷ dense	spread
6	`no_comm`	48	30	11	6	4	2.07	1.536	4.6
6	`dense`	49	29	11	6	4	2.16	1.596	6.6
6	`dense_sdpa`	75	7	4	11	2	1.35	1.000	31.5
6	`topk_masked`	71	7	6	13	3	1.42	1.049	7.8
6	`topk_gather`	78	8	4	8	2	1.31	0.967	25.5
12	`no_comm`	84	9	3	3	1	3.93	1.000	12.9
12	`dense`	85	11	2	2	1	3.95	1.005	8.9
12	`dense_sdpa`	85	9	2	3	1	3.93	1.000	11.1
12	`topk_masked`	76	14	6	2	2	4.48	1.138	29.8
12	`topk_gather`	75	14	5	3	2	4.58	1.165	1.7
24	`no_comm`	92	5	2	1	1	13.37	1.011	3.3
24	`dense`	92	5	2	1	1	13.37	1.011	2.5
24	`dense_sdpa`	91	5	2	1	1	13.23	1.000	1.7
24	`topk_masked`	91	5	2	1	1	13.44	1.016	2.1
24	`topk_gather`	91	5	2	1	1	13.26	1.003	3.6

realise part of it. What no kernel removes is that the score matrix must be formed and ranked at all, and the fused measurements of Section 4.3 show what remains once the headroom is taken.

4.6 Backward pass and training throughput

Every latency reported so far is a forward pass, which for a training claim is half the question. Timing forward and backward under the same protocol leaves the ordering intact and roughly a third of the margin (Table 4). The gathered path's backward costs about half its own forward, because the expensive stage — selection — has almost no adjoint: the retained indices are recorded in the forward pass and the backward is a scatter of gradients into them. Dense attention's backward costs several times its forward. At 512 agents the gathered path is 18.5 times the fused dense path on the forward alone and 6.3 times on forward and backward together; the same narrowing appears at every agent count. The sparse path's disadvantage in training is therefore substantially smaller than the forward-only measurements suggest, and remains a disadvantage. The fused kernels are excluded from this comparison: they are hand-written forward passes, and timing one against an autograd graph would compare two different things.

End to end, none of this is visible. Timing a complete training iteration by phase, with the communication implementation swapped and everything else fixed, puts the environment at 91% of the iteration at twenty-four agents and leaves all five implementations within 1.6% of one another, against an iteration-to-iteration spread of 1.7 to 3.6% (Table 5). The communication-free variant, which performs no attention at all, is not the fastest of the five — which is the clearest available demonstration that this measurement does not resolve

the implementations rather than that it resolves them as equal. We report it as the null it is. It bounds the practical stakes of the whole measurement campaign in the regime the training study occupies, and it is why the computational argument is made on the module in isolation: an effect a training loop cannot resolve is still an effect, but a paper claiming a throughput improvement from it would be claiming something it cannot measure.

4.7 Robustness

The ordering is unchanged under `torch.compile` and CUDA-graph capture, at `bf16`, and on an x86_64 CPU whose machine balance differs from the GPU's by more than an order of magnitude. The CPU comparison required controlling an effect unrelated to attention: with the default allocator the multi-MiB temporaries each implementation frees between calls are returned to the kernel and re-faulted on the next call, which at N=192 adds roughly 37,000 minor page faults per forward and inflates latency by up to 8.8×. The tax is proportional to the bytes of large temporaries each implementation churns, so it does not cancel in a ratio, and in an un-pinned run it inverted the ordering at one configuration. With the heap pinned the inversion disappears. Appendix B.4 gives the procedure and both runs.

Because clocks are unlocked and the host is shared, we re-ran the identical sweep with the device otherwise idle and again under a concurrent load. With the device idle the ratios reproduce to within a few per cent; under load absolute latency moves by more than 100% while the paired ratios move by roughly half that, and every ratio moves against the sparse path. Pairing removes the typical drift rather than the tail, so the quantity that matters is the margin: no condition we measured brings a composed sparse implementation below parity, and the fused crossover reported in Section 4.3 is measured on the quietest device state we could obtain. Appendix A.3 gives the figures.

5 Selecting Before Scoring

Proposition 2 says that a selector which fixes the retained set before any score exists is not bound by the twofold ceiling, and that its operation-count saving grows without limit as the retained set shrinks. That is a statement about counts, and the rest of this paper is about what counts fail to predict. This section instantiates the escape and measures it.

Two selectors, both chosen because multi-agent communication already uses mechanisms of their kind rather than because they are new. The first retains each agent's k nearest peers by Euclidean distance in the two-dimensional position space, which is what neighbourhood-based communication does (Jiang et al., 2020; Jiang & Lu, 2018); its selector is still pairwise, but pairwise in two dimensions rather than in the head dimension, so it costs a factor of $d/2$ less than forming the scores. The second orders agents along a Morton curve and retains the k agents adjacent in that order, which costs $O(N \log N)$ and so satisfies the $o(Nkd)$ condition of case (i) of the proposition for any fixed k. Both then evaluate attention scores for the retained pairs only, so the score term falls from $N^2 d$ to Nkd and the ceiling does not apply.

The two selectors attend to different peers than a top-k rule would, so what follows is a comparison of cost and not of quality. Selection and attention are timed separately as well as together, because a proposition that charges for the selector is only tested by an experiment that charges for it.

Both selectors deliver what the accounting promises and neither delivers a speedup, and the reasons differ in a way that identifies a second condition the count cannot express.

The nearest-neighbour selector removes an order of magnitude of arithmetic and gets slower, relative to dense attention, as the agent count rises: at 512 agents it performs roughly twenty times fewer operations than dense attention and takes fifteen times longer. Its selector is quadratic, so it materialises an $N \times N$ distance matrix and ranks it — the same two operations, on the same shape, that Proposition 1 says a score-then-select rule cannot avoid. Working in two dimensions rather than in the head dimension buys a factor in the arithmetic and nothing in the traffic, and Section 4.5 shows traffic to be what governs. A pre-score selector of this kind escapes the proposition on paper and reproduces its cost structure in silicon.

Table 6: Selecting before scoring, at 4 retained peers. The operation-count saving is large and grows with the agent count; the elapsed time does not follow it. Selection and attention are timed separately and the total is timed as one call, so the components need not sum exactly.

		operations	**measured (ms)**				
N	selector	saved (×)	select	attend	total	dense	ratio
48	nearest neighbours	7.7	0.049	0.030	0.078	0.011	7.17
48	Z-order window	10.0	0.162	0.030	0.197	0.011	18.19
96	nearest neighbours	11.3	0.098	0.046	0.144	0.027	5.29
96	Z-order window	19.8	0.174	0.046	0.226	0.027	8.32
192	nearest neighbours	14.8	0.317	0.087	0.397	0.066	6.01
192	Z-order window	39.4	0.176	0.087	0.271	0.066	4.10
384	nearest neighbours	17.5	1.812	0.166	1.984	0.216	9.20
384	Z-order window	78.3	0.189	0.165	0.353	0.216	1.64
512	nearest neighbours	18.3	5.224	0.221	5.514	0.345	16.00
512	Z-order window	103.7	0.197	0.221	0.437	0.345	1.27

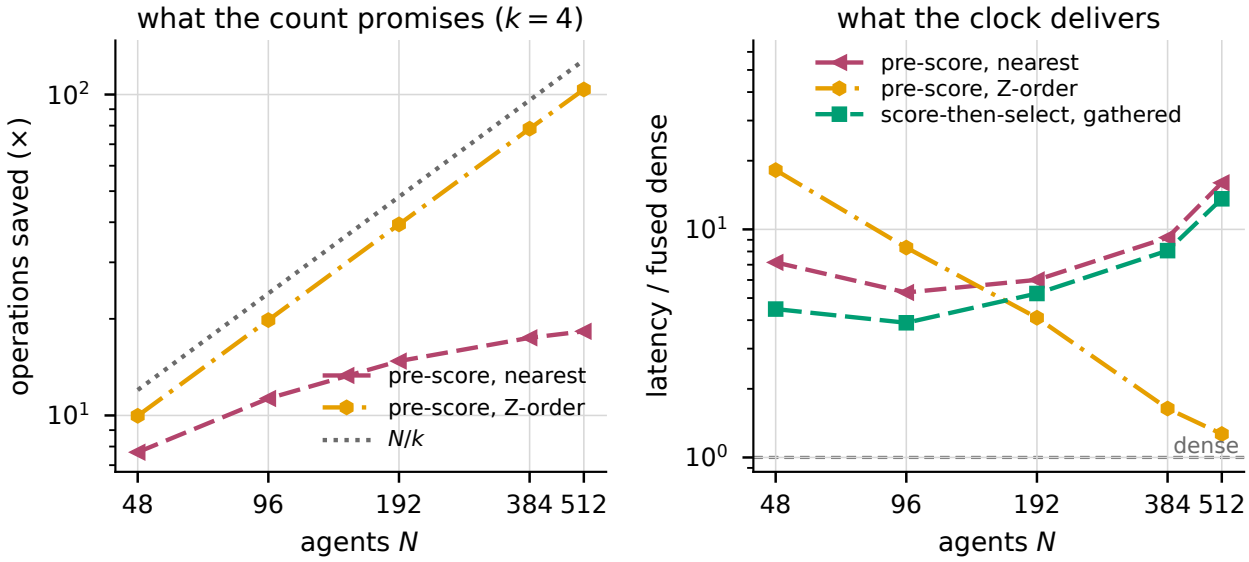


Figure 4: **Left:** the operation-count saving Proposition 2 promises, which for the sub-quadratic selector tracks N/k. **Right:** what the same configurations cost. Escaping the ceiling in operation count does not by itself produce a speedup, and the two selectors that the count treats similarly behave oppositely as the agent count grows.

The Morton selector behaves in the opposite direction and comes close. Its cost is essentially independent of the agent count over our whole range, because a sort of a few hundred keys per batch element is bound by kernel launch rather than by work, and its attention stage — scoring and aggregating only the retained pairs — is roughly twice as fast as dense attention at the largest agent count, which is what Proposition 2 predicts once the score term shrinks with the aggregation. The ratio against dense attention therefore improves monotonically with the agent count, from an order of magnitude at 48 agents to within a small margin of parity at 512. What keeps it above parity is not the attention at all but the fixed cost of the sort, which at these sizes is comparable to the entire dense attention it is trying to displace.

Three conclusions follow, and the third is the one we would have missed without this measurement.

Escaping the operation-count ceiling is necessary. The Morton selector's attention stage is genuinely faster than dense attention, and no score-then-select implementation in this paper achieves that at any comparable sparsity, because none of them can remove the score term.

Escaping it is not sufficient, and the accounting says nothing about which of the two selectors will be faster. Both satisfy the proposition; one improves with scale and one deteriorates. What separates them is whether the selector reaches its decision through a structure of the size it was introduced to avoid. Proposition 2 charges for the selector in operations, and operations are the wrong currency: the nearest-neighbour selector's arithmetic is small and its traffic is exactly the traffic the ceiling is about.

And a selector must be cheap in absolute terms at the size in question, not merely asymptotically. The Morton selector is asymptotically free relative to attention and is nonetheless the reason its path does not win, because a few hundred agents is small enough that a launch-bound sort costs as much as the quadratic work it is avoiding. This is the same quantity that makes block-level selection succeed in long-context attention, where a selector amortised over blocks of a long sequence is negligible against the work it skips (Yuan et al., 2025; Lai et al., 2026), and it is the quantity a successor method for multi-agent communication has to get right. Nothing about the analysis suggests this is impossible; it does suggest that the design target is the selector's constant, not the aggregation's asymptotics.

6 Learning Study

Sparsification would still be worth having if it cost nothing in return. Testing that requires care, because on a task where the channel is not needed "sparsification is free" and "the channel was never used" make identical predictions. We therefore gate the study on a positive control and register the analysis plan before the outcomes are inspected. The study is secondary. Its role is to establish that the architecture being sparsified does something, so that the sparsification comparisons are not vacuous, and to report what a five-seed design can and cannot resolve.

6.1 Design

Twelve variants at up to twelve agents, five seeds each, two million environment steps. All scaling in the number of agents is delegated to the training-free measurements of Section 4, because an end-to-end training step in this pipeline is dominated by the environment (Section 4.6) and cannot attribute a latency difference to the communication module.

Two positive controls establish that the channel carries something. The first is a reference task in which each agent observes only its partner's goal, so information must pass between agents; it is fixed at two agents, where the selection mechanism is degenerate, so it validates the message pathway rather than the selection. The second is a partially observable variant of a standard cooperative task, introduced here, in which a peer's relative position is withheld beyond a fixed radius. Under a uniform random policy the fraction of peers visible falls with the number of agents while the absolute number visible rises, because the world does not grow: that is a difficulty confound across agent counts, and it is why no claim in this paper compares returns across agent counts in this environment.

The analysis plan was committed after eight of the forty runs in the pre-registered family had completed and before any of their outcomes was inspected; the two positive controls were complete and had been inspected, which is why they are excluded from the inferential family. Appendix C.1 gives the chronology and the single amendment. The plan fixes the endpoint, the six comparisons and their direction, an exact two-sided permutation test, the effect sizes, a percentile bootstrap interval flagged approximate at five seeds, and a Holm–Bonferroni correction. It also fixes the power expectation in advance: with five seeds per variant the smallest attainable two-sided p-value is $2/252 = 0.0079$ against a first Holm threshold of $0.05/6 = 0.0083$, a margin of 4×10^{-4}, so a first rejection requires one variant's five seeds to lie entirely above the other's. Any null in this family is under-powered rather than evidence of absence, and no equivalence claim is available because no equivalence margin was registered.

6.2 Results

Both positive controls separate completely, each attaining the floor of the permutation test for this design (Table 7). Measured attention entropy on the partially observable control is 0.35 nats against a ceiling of

Table 7: The two positive controls, which were inspected before registration and are excluded from the correction, and the primary pre-registered comparison. $A-B$ is the difference of means and d the standardised effect. Appendix C gives the remaining five comparisons.

	$A-B$	95% interval	d	p
Positive controls, descriptive				
communicating − non-communicating, reference task	+4.26	[+3.15, +5.23]	—	0.0079
communicating − non-communicating, partially observable	+11.59	[+7.13, +16.44]	—	0.0079
Primary pre-registered comparison				
attention-selected peers − uniformly random peers, same budget	+4.03	[−3.35, +11.42]	+0.60	0.357

1.61, so the learned weighting is peaked rather than uniform. What this establishes is that sparsification is being applied to an architecture whose communication machinery improves return under the evaluated setup. It does not establish that the learned attention channel alone accounts for the difference: the communicating variants carry 7–19% more trainable parameters, and the reference task supplies a discrete utterance channel of its own. A parameter-matched sham-communication variant would be needed for the channel-specific causal claim, and we do not make it.

The comparison the design exists for holds the communication budget, the executed aggregation cost and every hyperparameter fixed, and varies only whether peers are chosen by attention score or uniformly at random. Its point estimate favours attention by roughly two thirds of a standard deviation and its interval includes zero. The design lacked the power to resolve an effect of that size, which the pre-registered power calculation said in advance; nothing follows about whether attention selects informatively. The remaining five comparisons are reported in Appendix C as point estimates with intervals (Agarwal et al., 2021), and all six are Holm-adjusted nulls. No conclusion in this paper rests on any of them.

Two limits of the environment bound what the study can say. At twelve agents neither variant learns the task well: the dense variant scores below a uniform-random policy and the sparse variant only just above it, so no comparison at that agent count is interpretable. And the centralised critic sees the same masked observations as the actors rather than a privileged global state, which departs from standard practice for this learner and is a plausible contributor to that failure.

7 Sources of Measurement Error

Every measurement in this paper was at some point wrong, and in each case the first symptom was a plausible number rather than an obvious error. The causes group into four kinds, and the grouping is useful because the kinds differ in how they are caught. Table 8 lists them with the corrective practice each forced.

Instrumentation. A diagnostic that synchronises the device inside the timed region measures the synchronisation. Our attention module computed an entropy statistic through a call that transfers a scalar to the host, and with it enabled dense latency was nearly flat in N while its work grew by two orders of magnitude — a result we briefly read as evidence that the quadratic cost does not manifest. Synchronising without naming the device is the complementary error: work submitted to a non-default device is never waited on, and every kernel at every size then reports the event-resolution floor of about 0.4 μs. On the host, the default allocator returns the multi-MiB score matrices each implementation frees between calls, so the next call re-faults them; at N=192 that added roughly 37,000 minor page faults per forward and inflated latency by up to 8.8×. Errors of this kind leave traces — an impossible latency, a curve that is flat where it should be quadratic, a non-monotone cost — and are caught by looking at the shape of the result rather than its value.

Interference from a shared device. Timing implementations in sequential blocks makes each block hostage to whatever else was running during it. On our shared host that produced swings above 100% between identical configurations and a 3.7× discrepancy for one implementation at one agent count. Interleaving

the implementations repetition by repetition lands any interference burst on all of them at once, so the ratios between them survive even when the absolute numbers drift. The residual sensitivity is to which implementations share an interleaved sweep: at small N, where cost is governed by kernel launches rather than by arithmetic or bandwidth, the launch count of the whole group enters every measurement in it, and two of our own sweeps disagree by 5× at one such configuration. Sweeps that will be compared must therefore hold their implementation set fixed and say so.

Measuring a different algorithm from the one claimed. The two most dangerous errors are of this kind, because the resulting numbers are internally consistent, reproducible, and stable under every check the others would fail. They answer a different question from the one being asked. Applying top-k as a mask leaves the dense aggregation matmul in place, so the advertised saving is never executed; a FLOP model based on $1 - k/(N-1)$ nonetheless credits that code with a 75% saving at N=512, k=127, and with more than 90% wherever $k/(N-1) < 0.1$. Comparing against unfused dense alone is the same error in the baseline: at N=512 the unfused and fused dense paths differ by 4.6×, so which one is meant sometimes changes the answer. The defence is to state the operation count each implementation executes alongside its latency, and to check which backend actually serves each shape and dtype, both of which make a mismatch between the claimed and the executed algorithm visible immediately.

Attributing implementation quality to the algorithm. The fourth kind appeared only when we wrote our own kernels, and it is the one we would most readily have published. Timing a hand-written sparse kernel against a vendor library's dense kernel measures sparsity and authorship at once. At the head widths used in multi-agent communication, PyTorch's fused attention path runs at roughly half the rate of a straightforward Triton kernel we wrote in an afternoon, because it is tuned for the long sequences and wide heads of language models. Compared against it, our fused sparse kernel appears up to 2.5× faster than dense attention — a speedup that exceeds what Proposition 1 permits from sparsity, which is the signal that something other than sparsity is producing it. Against a dense kernel written in the same framework with the same tiling, the same measurement shows a much smaller advantage, confined to the smallest k, and consistent with the ceiling. A sparse implementation must be compared against a dense implementation of matched engineering effort, or the comparison silently credits sparsity with the difference between two authors.

One belief of our own also needed correcting. We assumed that standard counters are blind to fused attention kernels, which would make a FLOPs-only comparison unable to see the baseline that matters. In PyTorch 2.11 that is not so: `torch.utils.flop_counter` reports the fused path correctly, and it was our harness that added an analytic term on top of the counter's output for that implementation, publishing a figure 1.83× too large. The narrower point generalises better than the one we started with: a counter has to be validated against the kernels it is expected to observe, because only a counter attached to the running program will contradict a FLOP model that is describing code nobody executed.

8 Limitations

Every cost result here concerns centralised inference on a single device: the latency and memory of executing the communication module in one process that holds all agents' observations. It is not a claim about bandwidth-constrained distributed deployment, where the objective is to reduce messages physically transmitted and top-k sparsification may well be correct even though it does not reduce local computation. Proposition 1 says nothing about transmitted bytes, and neither do the measurements. The analysis is also single-round: assumptions A1 and A3 show that the operation-count *ratio* is unchanged by additional rounds or heads, but the measurements are of one round of one head, and at the smaller agent counts we find cost governed by kernel launches and cache residency, both of which multi-round communication changes.

The measured envelope is one GPU architecture (NVIDIA Blackwell, sm_120), one x86_64 CPU, PyTorch 2.11 with CUDA 12.8 and Triton 3.6, scaled dot-product attention through the fused backends that build provides, input embedding width 64, head width $d \in \{16, \ldots, 512\}$, $N \leq 512$, batch 32–1024, `fp32` and `bf16`, and eager, compiled and CUDA-graph execution. We were unable to replicate on a different GPU generation: the host has two cards of the same model, and the older hardware that produced this project's earlier

Table 8: Eight ways a benchmark measures something other than the algorithm, each of which produced a spurious result in this study. The first five are caught by the shape of the result; the last three are not, because the numbers they produce are self-consistent.

Kind	**Cause**	**Symptom**	**Corrective practice**
Instrumentation	Host transfer inside the timed region	Dense latency nearly flat in N	Gate diagnostics behind a flag that is off while timing
	Synchronisation without a device argument	Every kernel reports the event-resolution floor	Pin the device and pass it to every synchronisation
	Host allocator returning large temporaries	Up to 8.8× inflation and a non-monotone curve	Pin the heap; count minor faults per forward
Interference	Timing implementations in sequential blocks	Swings above 100% between identical configurations	Interleave repetition by repetition; report paired ratios
	Changing which implementations share a sweep	Two sweeps disagree by 5× at one configuration	Fix the implementation set across comparable sweeps
Wrong algorithm	Benchmarking a masked implementation as sparse	The advertised saving is never executed	Assert executed FLOPs differ and trace the products issued
	Comparing only against unfused dense	A 4.6× more favourable comparison at N=512	Report both baselines; check which backend serves the shape
Wrong baseline	A hand-written sparse kernel against a vendor's dense kernel	An apparent speedup larger than the analytic ceiling permits	Write the dense baseline in the same framework and tiling

runs is no longer available. The arithmetic-intensity argument of Section 3.4 predicts the same qualitative behaviour wherever the relevant machine balance is similar, but an argument that is device-independent by construction cannot itself establish device-independence, so generality across GPU generations is untested. Both cards were shared with other tenants throughout, which is why every claim is stated as a ratio between implementations timed in the same interleaved sweep, and why the drift control is a same-device repeat rather than a second card.

Our sparse implementations are not optimal and we can bound by how much in two directions rather than one. The composed PyTorch path leaves substantial performance on the table: removing an unnecessary sort was worth 5–24%, and the remaining selection kernel runs at roughly 9% of copy-equivalent bandwidth. The fused kernels remove most of what remains, and they are the reason the manuscript's earlier unconditional statement — that the sparse path is slower in every measured configuration — does not survive. They are in turn not the last word: they are written in Triton rather than tuned assembly, their selection strategies are argmax rounds and threshold bisection rather than a radix or bitonic select, they hold the whole score row in registers and so are capped at $N \leq 1024$, and they are forward-only, which is why the backward comparison covers the composed implementations alone. A better kernel would narrow the remaining gap further. What no kernel can remove is the requirement to form and rank the full score matrix, which is assumption A2, and the operation-count ceiling that follows from it.

The learning study is secondary and bounded by its power. Five seeds per implementation gives a minimum attainable two-sided permutation p of 0.0079 against a first Holm threshold of 0.0083, so any rejection in the pre-registered family requires the smallest p to sit at the discrete floor. Resolving the $d \approx 0.6$ effect the primary contrast suggests would need roughly 45 seeds per implementation, or 70 at the family-corrected level, both from a normal-approximation formula that guides order of magnitude rather than describing the exact permutation test we run. That was a budget allocation rather than an unaffordable experiment: the whole 60-run study consumed 31.9 GPU-hours and a properly powered primary contrast would have cost about 37.9 more. The positive controls establish that the communicating architecture improves return rather than isolating the learned channel, since the communicating variants carry 7–19% more trainable

parameters and one of the two control tasks supplies a communication channel of its own; a parameter-matched sham-communication variant would be needed for the channel-specific causal claim, and we do not make it. Our partially observable environment carries a difficulty confound across agent counts that we measure and publish, which is why no across-N return comparison appears, and its centralised critic sees the same masked observations as the actors rather than a privileged global state, which departs from standard practice and is a plausible contributor to the failure of both variants to learn well at twelve agents.

Finally, the pre-score selectors of Section 5 demonstrate a cost structure, not a method. They attend to a different peer set than top-k does, so the comparison against it is a comparison of cost and not of quality, and we make no claim that spatial neighbours carry the information attention would have selected. Whether a pre-score selector preserves task performance is the obvious next question and one this paper does not answer.

9 Conclusion

The efficiency case for restricting inter-agent attention to each agent's highest-scoring peers rests on an operation count, and the count is smaller than it looks. Because the selection reads the scores, every pairwise interaction has been evaluated before any is discarded, so only the aggregation is sparsifiable and no choice of sparsity removes more than half of attention's arithmetic. That bound is not a property of top-k but of the price of the selector: writing it as one accounting in which a rule is charged for what it costs to decide places every restriction mechanism on a single axis, and the mechanisms that obtain asymptotic savings are exactly those whose selectors are cheap.

Measuring what executes shows the bounded saving to be optimistic twice over. An implementation that applies the selection as a mask performs the dense aggregation anyway, so it executes no saving at all while a count credits it with a large one. An implementation that genuinely compacts the retained peers executes the saving and is slower regardless, because compaction replaces multiplication by zeros with the writing and re-reading of a gathered tensor, and at these sizes time follows bytes. The criterion that emerges is simple enough to check before writing the code: gathering is byte-neutral only when the compacted operand is no larger than the score row it avoids re-reading.

Whether any of this is a property of the algorithm or of the tools was the question our earlier measurements could not answer, and writing the kernels answered it in both directions. Most of the measured penalty is the composition of general-purpose operators, and a fused kernel removes it. What remains is the algorithm, and it is decisive at the sparsity levels this literature uses: the fastest sparse implementation we could write is faster than the fastest dense implementation we could write only when one or two peers are retained, by a margin the analytic ceiling bounds, and is an order of magnitude slower once a realistic number of peers is kept. Two structural reasons survive every optimisation we tried. Ranking N candidates to keep k of them costs more, on the units that perform the ranking, than aggregating over the k saves; and a kernel can make that ranking cheap in k only by expressing the retained set as a zero weight, which abandons the sparse arithmetic it was built to obtain. Exact score-then-select attention, fused, can have sparse arithmetic or cheap selection, and we could not obtain both.

The constructive implication is not that sparse communication is a bad idea but that the saving has to be taken before the scores exist rather than after. A selector that fixes the retained set in advance is not bound by the ceiling, and its promised saving grows without limit as the retained set shrinks. Measuring two such selectors shows that removing the bound is necessary and not sufficient, and shows it twice over. A selector that reaches its decision through a quadratic intermediate — ranking a matrix of pairwise distances rather than of pairwise scores — delivers an order of magnitude of arithmetic saving and is an order of magnitude slower, because it reproduces in two dimensions exactly the traffic the ceiling was about. A selector that never forms such a structure delivers the saving where it matters: its attention stage is roughly twice as fast as dense attention at the largest agent count we measure, which no score-then-select implementation in this paper achieves at any sparsity. Even that one does not win overall, and the reason is the third finding: its own fixed cost, a sort of a few hundred keys, is comparable at these sizes to the entire dense attention it is displacing.

A successor method is therefore accountable for three things, of which an operation count shows only the first: what its selector costs in operations, whether it reaches its decision without materialising a structure of the size it means to avoid, and whether its constant is small against the work it skips at the size in question. The third is why block-level selection succeeds in long-context attention and why the same idea does not transfer unchanged to a few hundred agents: a selector amortised over the blocks of a hundred-thousand token sequence is negligible against what it skips, while the same selector against a few hundred agents is not. That is a statement about a ratio rather than about a mechanism, and it is checkable in advance, which is the most useful thing this analysis produces.

Broader Impact Statement

This work is a measurement and correction study of a computational-efficiency claim. It introduces no new capability and no new data. Its intended effect is to reduce wasted compute in the design of multi-agent communication architectures by discouraging optimisation against an operation count that does not predict runtime.

References

Joshua Achiam, David Held, Aviv Tamar, and Pieter Abbeel. Constrained policy optimization. *International Conference on Machine Learning (ICML)*, pp. 22–31, 2017. URL https://proceedings.mlr.press/v70/achiam17a.html.

Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. URL https://arxiv.org/abs/2108.13264.

Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document Transformer. *arXiv preprint*, 2020. URL https://arxiv.org/abs/2004.05150.

Krzysztof Choromanski, Valerii Likhosherstov, David Dohan, Xingyou Song, Andreea Gane, Tamas Sarlos, Peter Hawkins, Jared Davis, Afroz Mohiuddin, Lukasz Kaiser, David Belanger, Lucy Colwell, and Adrian Weller. Rethinking attention with Performers. *International Conference on Learning Representations (ICLR)*, 2021. URL https://arxiv.org/abs/2009.14794.

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-Awareness. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://proceedings.neurips.cc/paper_files/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract-Conference.html.

Abhishek Das, Théophile Gervet, Joshua Romoff, Dhruv Batra, Devi Parikh, Michael Rabbat, and Joelle Pineau. TarMAC: Targeted multi-agent communication. *International Conference on Machine Learning (ICML)*, 2019. URL https://arxiv.org/abs/1810.11187.

William Fedus, Barret Zoph, and Noam Shazeer. Switch Transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022. URL https://jmlr.org/papers/v23/21-0998.html.

Ankit Gupta, Guy Dar, Shaya Goodman, David Ciprut, and Jonathan Berant. Memory-efficient Transformers via top-k attention. In *Proceedings of the Second Workshop on Simple and Efficient Natural Language Processing (SustaiNLP)*, pp. 39–52. Association for Computational Linguistics, November 2021. doi: 10.18653/v1/2021.sustainlp-1.5. URL https://aclanthology.org/2021.sustainlp-1.5/.

Mohammed Hamdan, Fares Ghaleb, Ahmed Bali, and Mohamed Cheriet. MARL-IoTP: Heterogeneous multi-agent learning for perception-aware edge orchestration. In *Fifth International Conference on Pattern Recognition and Artificial Intelligence (PRAI)*, 2026. doi: 10.2139/ssrn.6517702. URL https://ssrn.com/abstract=6517702. Accepted.

Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with Gumbel-Softmax. *International Conference on Learning Representations (ICLR)*, 2017. URL https://arxiv.org/abs/1611.01144.

Jiechuan Jiang and Zongqing Lu. Learning attentional communication for multi-agent cooperation. *Advances in Neural Information Processing Systems (NeurIPS)*, 2018. URL https://arxiv.org/abs/1805.07733.

Jiechuan Jiang, Chen Dun, Tiejun Huang, and Zongqing Lu. Graph convolutional reinforcement learning. *International Conference on Learning Representations (ICLR)*, 2020. URL https://arxiv.org/abs/1810.09202.

Daewoo Kim, Sangwoo Moon, David Hostallero, Wan Ju Kang, Taeyoung Lee, Kyunghwan Son, and Yung Yi. Learning to schedule communication in multi-agent reinforcement learning. In *International Conference on Learning Representations (ICLR)*, 2019. URL https://openreview.net/forum?id=SJxu5iR9KQ.

Nikita Kitaev, Łukasz Kaiser, and Anselm Levskaya. Reformer: The efficient Transformer. *International Conference on Learning Representations (ICLR)*, 2020. URL https://arxiv.org/abs/2001.04451.

Xunhao Lai, Weiqi Xu, Yufeng Yang, Qiaorui Chen, Yang Xu, Lunbin Zeng, Xiaolong Li, Haohai Sun, Haichao Zhu, Vito Zhang, Jinkai Hu, Jiayao Li, Rui Gao, Zekun Li, Songquan Zhu, Jingkai Zhou, and Pengyu Zhao. MiniMax sparse attention. *arXiv preprint*, 2026. doi: 10.48550/arXiv.2606.13392. URL https://arxiv.org/abs/2606.13392.

Yen-Cheng Liu, Junjiao Tian, Chih-Yao Ma, Nathan Glaser, Chia-Wen Kuo, and Zsolt Kira. Who2com: Collaborative perception via learnable handshake communication. In *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 6876–6883, 2020a. doi: 10.1109/ICRA40945.2020.9197364.

Yong Liu, Weixun Wang, Yujing Hu, Jianye Hao, Xingguo Chen, and Yang Gao. Multi-agent game abstraction via graph attention neural network. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 34, pp. 7211–7218, 2020b. doi: 10.1609/aaai.v34i05.6211.

André F. T. Martins and Ramón Fernandez Astudillo. From softmax to sparsemax: A sparse model of attention and multi-label classification. *International Conference on Machine Learning (ICML)*, 2016. URL https://arxiv.org/abs/1602.02068.

Yaru Niu, Rohan R. Paleja, and Matthew C. Gombolay. Multi-agent graph-attention communication and teaming. In *Proceedings of the 20th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, pp. 964–973, 2021. URL https://www.ifaamas.org/Proceedings/aamas2021/pdfs/p964.pdf.

Georgios Papoudakis, Filippos Christianos, Lukas Schäfer, and Stefano V. Albrecht. Benchmarking multi-agent deep reinforcement learning algorithms in cooperative tasks. *Advances in Neural Information Processing Systems Datasets and Benchmarks Track (NeurIPS)*, 2021. URL https://arxiv.org/abs/2006.07869.

Ben Peters, Vlad Niculae, and André F. T. Martins. Sparse sequence-to-sequence models. *Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 1504–1519, 2019. doi: 10.18653/v1/P19-1146. URL https://aclanthology.org/P19-1146/.

Markus N. Rabe and Charles Staats. Self-attention does not need $O(n^2)$ memory. *arXiv preprint*, 2021. URL https://arxiv.org/abs/2112.05682.

Aurko Roy, Mohammad Saffar, Ashish Vaswani, and David Grangier. Efficient content-based sparse attention with Routing Transformers. *Transactions of the Association for Computational Linguistics (TACL)*, 9: 53–68, 2021. doi: 10.1162/tacl_a_00353. URL https://aclanthology.org/2021.tacl-1.4/.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *International Conference on Learning Representations (ICLR)*, 2017. URL https://arxiv.org/abs/1701.06538.

Amanpreet Singh, Tushar Jain, and Sainbayar Sukhbaatar. Learning when to communicate at scale in multiagent cooperative and competitive tasks. *International Conference on Learning Representations (ICLR)*, 2019. URL https://arxiv.org/abs/1812.09755.

Adam Stooke, Joshua Achiam, and Pieter Abbeel. Responsive safety in reinforcement learning by PID Lagrangian methods. *International Conference on Machine Learning (ICML)*, pp. 9133–9143, 2020. URL https://proceedings.mlr.press/v119/stooke20a.html.

Sainbayar Sukhbaatar, Arthur Szlam, and Rob Fergus. Learning multiagent communication with backpropagation. *Advances in Neural Information Processing Systems (NeurIPS)*, 2016. URL https://arxiv.org/abs/1605.07736.

Chuangchuang Sun, Macheng Shen, and Jonathan P. How. Scaling up multiagent reinforcement learning for robotic systems: Learn an adaptive sparse communication graph. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 11755–11762, 2020. doi: 10.1109/IROS45743.2020.9341303. URL https://arxiv.org/abs/2003.01040.

J. K. Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis Santos, Clemens Dieffendahl, Caroline Horsch, Rodrigo Perez-Vicente, Niall L. Williams, Yashas Lokesh, and Praveen Ravi. PettingZoo: Gym for multi-agent reinforcement learning. *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. URL https://arxiv.org/abs/2009.14471.

Muning Wen, Jakub Grudzien Kuba, Runji Lin, Weinan Zhang, Ying Wen, Jun Wang, and Yaodong Yang. Multi-agent reinforcement learning is a sequence modeling problem. *Advances in Neural Information Processing Systems (NeurIPS)*, 2022. URL https://arxiv.org/abs/2205.14953.

Samuel Williams, Andrew Waterman, and David Patterson. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785. URL https://doi.org/10.1145/1498765.1498785.

Yaodong Yang, Rui Luo, Minne Li, Ming Zhou, Weinan Zhang, and Jun Wang. Mean field multi-agent reinforcement learning. *International Conference on Machine Learning (ICML)*, 2018. URL https://arxiv.org/abs/1802.05438.

Chao Yu, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen, and Yi Wu. The surprising effectiveness of PPO in cooperative, multi-agent games. *Advances in Neural Information Processing Systems Datasets and Benchmarks Track (NeurIPS)*, 2022. URL https://arxiv.org/abs/2103.01955.

Jingyang Yuan, Huazuo Gao, Damai Dai, Junyu Luo, Liang Zhao, Zhengyan Zhang, Zhenda Xie, Y. X. Wei, Lean Wang, Zhiping Xiao, Yuqing Wang, Chong Ruan, Ming Zhang, Wenfeng Liang, and Wangding Zeng. Native sparse attention: Hardware-aligned and natively trainable sparse attention. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 23078–23097. Association for Computational Linguistics, 2025. URL https://aclanthology.org/2025.acl-long.1126/.

Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big Bird: Transformers for longer sequences. *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. URL https://arxiv.org/abs/2007.14062.

A Additional benchmark results

A.1 The full parameter grid

Table 9 gives the best-over-implementations sparse latency divided by the best-over-implementations dense latency, across head width and across batch, at each agent count and at the most favourable sparsity available. Head width is swept to 512, which resolves a trend the earlier version of this manuscript left open at 256;

batch is swept from 32 to 1024. The largest batch we attempted, 4096 at 512 agents, could not be measured: the score matrix alone runs to several gigabytes and the device is shared, so that configuration is absent rather than reported.

Table 9: Best sparse divided by best dense, over head width (upper block) and batch (lower block). Values below one would indicate sparsity winning.

head width d	N=96	N=192	N=512
16	0.66	0.71	0.74
32	0.51	0.69	0.62
64	0.87	1.32	2.15
128	1.38	2.17	1.88
256	2.10	2.07	2.11
512	1.66	1.72	2.18

batch B	N=96	N=192	N=512
32	0.55	0.65	0.57
1024	0.55	0.75	0.72

A.2 Dispersion behind the headline numbers

The primary statistic is the minimum over interleaved samples, on the modelling assumption that contention can only add time. That statistic is trustworthy only if the underlying sample was not wildly dispersed, so the harness records the median, the interquartile range and the spread of per-run medians for every configuration, and the released artifacts carry all of them. Two properties of this host bound what any of it can mean. Both GPUs carried between 30 and 80 GiB of other tenants' allocations and near-continuous utilisation throughout the campaign, and clocks could not be locked. We therefore report the spread of per-run medians beside every configuration rather than a confidence interval: a ratio of minima has no sampling distribution, and treating it as though it did would be the same error the rest of this paper is about.

A.3 Drift control

The identical sweep was repeated on the same device, once with the device as idle as we could obtain it and once under a concurrent load. With the device idle the ratios reproduce closely; under load absolute latency moves by more than 100% while paired ratios move by roughly half as much, and every ratio moves against the sparse path. Pairing removes the typical drift rather than the tail. This is a same-device repeat, so it establishes stability across runs and hours, not independence from one physical card.

B Implementation and measurement validation

B.1 Functional equivalence

Every implementation is checked against the reference at every configuration whose latency is reported. The check matters more than it looks: exact top-k is discontinuous in the scores, so two implementations that accumulate the score product in different orders can rank a near-tied pair differently and aggregate over different peers, producing a large output difference from a rounding-level score difference. An earlier version of our per-query fused kernel computed the score row as a reduction rather than through the matrix-multiply units, and disagreed with the reference at a small number of configurations for exactly that reason; the kernels reported here use the same product as the reference.

The fused kernel that selects by threshold retains every peer at or above the k-th largest score, so an exact tie at that boundary retains $k+1$ peers where a rank-based selection retains k. Measured incidence at N=512, k=64, d=64 over twelve seeds: one query row in 196,608. Every other row agrees with the reference to `fp32` tolerance.

B.2 What each implementation executes

The masked and gathered implementations compute the same function while executing different work, which is the distinction the paper turns on, so it is pinned by regression tests rather than asserted. Maximum absolute deviation between the two paths is below 10^{-10} in `float64`. A FLOP counter attached to the running program reports identical totals for dense and masked — the counter scores matrix products, and masked issues the same two as dense — against a smaller total for gathered, the difference being exactly the aggregation term. Tracing every matrix product confirms that the gathered path issues no dense aggregation product and does issue the compacted one, and that every top-k variant still issues the full score product, which is assumption A2 made mechanical.

B.3 Which backend serves the fused dense baseline

A probe forces each scaled-dot-product backend in turn, at both dtypes and at three agent counts, under no mask, an additive float mask and a Boolean mask, and the profiler records which kernel the default dispatcher launches. The FlashAttention backend is rejected under any non-null mask in either form, and in `fp32` under any input, so the library dense baseline runs the memory-efficient CUTLASS kernel. Forcing FlashAttention on the same unmasked `bf16` shape is faster than the kernel we do get, so a baseline able to reach it would widen rather than narrow the gap we report — which is part of why the dense baseline in this revision is the better of the library path and a kernel we wrote.

B.4 The host allocator control

Dense, masked and gathered each allocate and free several multi-MiB score matrices per forward. The default allocator returns those pages to the kernel and they are re-faulted on the next call, which at N=192 adds roughly 37,000 minor page faults per forward and inflates latency by up to 8.8×. The tax is proportional to the bytes of large temporaries each implementation churns, so it differs per implementation and does not cancel in a ratio; in an un-pinned run it inverted the ordering at one configuration. We raise the allocator's retention threshold so that freed temporaries are kept, and count minor faults per forward from the process's own resource usage across the timed region. The harness records the allocator environment, the thread counts, the affinity mask and the fault totals in each run's metadata, and the superseded un-pinned run is retained beside the pinned one.

B.5 Machine balance

Peak arithmetic rate is measured with a large matrix multiply and peak bandwidth with a large copy, in the same process as the measurements they normalise. The copy source is written before it is read: an untouched allocation on this operating system is backed by the shared zero page, so a host-side copy from it re-reads one cache line and reports a bandwidth several times the machine's real figure. An earlier version of this probe made that error, and every CPU ridge point derived from it was wrong in the direction that flattered the argument. The corrected probe reports two ridge points per device, one with a cache-resident buffer and one with a buffer larger than the last-level cache, because the two differ by a factor of two on the host and a claim that holds for one need not hold for the other.

C Additional training analysis

C.1 Pre-registration chronology

The analysis plan was committed when eight of the forty runs in the pre-registered family had produced a final evaluation and before any of their returns had been inspected. Commit history establishes the ordering: the positive-control artifacts at 12:18, the plan at 12:56, and the remaining artifacts at 19:11 the same day. The plan has two commits, the second appending to its deviations log; the endpoint, comparisons, test, effect sizes, interval and correction are byte-identical to the pre-unblinding version. Timestamps establish

ordering and nothing more, so the plan's independence is a claim about our conduct rather than something the artifact proves, and data generation had demonstrably already begun.

One correction is logged against the released plan. It asserts that at most one comparison in the family can reach corrected significance. That is false: Holm is a step-down procedure, so once a first rejection is obtained its later thresholds are looser and further rejections remain possible. The manuscript states the corrected version; the deviation is recorded rather than the artifact silently edited.

C.2 The remaining comparisons

Five further comparisons were registered and all are Holm-adjusted nulls. Two deserve a note rather than an interpretation. The comparison at twelve agents has the one sign opposite to its registered direction and should be set aside for a reason independent of its p-value: neither variant learns the task there. A uniform-random policy scores -758.9 under a forty-episode evaluation, against -864.9 for the dense variant, which is worse, and -717.9 for the sparse variant, which is better but barely learning; the comparison is between a diverged run and a struggling one. Separately, for two comparisons the bootstrap interval excludes zero while the permutation test does not. The registration named the permutation test, so it governs; the disagreement is unsurprising at five seeds, where the interval is coarse, discrete and of uncertain coverage. We report both so that the estimated effect and the strength of evidence for it are both visible.

Descriptively, and supporting no claim: the budgeted variant meets its budget but not by the intended route, its gate collapsing to an expected support near one within the first tenth of training while the dual multiplier decays to zero; the entmax variant settles at a support of 1.64 of five peers; and attention entropy falls from 0.35 nats over five peers to 0.12 over two. The three sit on different supports and are reported only as an instrumentation check.

C.3 The partially observable environment

The variant modifies the standard cooperative task in one respect: a peer's relative-position entry is zeroed when the peer is farther than a fixed radius. Reward, action space, landmark observations, episode length and observation dimension are unchanged. Two properties bound what it can support. The mask is a zeroing rather than an explicit visibility flag, so a peer at relative position exactly zero is not distinguishable from an absent one; measured under a uniform random policy, this affects at most 0.15% of visible peers. And the visible fraction falls with the number of agents while the absolute number visible rises, because the world size is fixed, which is a difficulty confound that rules out across-agent-count return comparisons.

The communicating variants carry between 7.2% and 19.0% more trainable parameters than the communication-free baseline, since the attention module adds parameters the baseline has no analogue for. The positive controls therefore establish that the communicating architecture outperforms the baseline, not that the channel alone accounts for the difference.

D Literature audit

For each audited paper we read the full text, including appendices and the official supplement where one exists, searched it for seven temporal keywords, and recorded whether it makes a computational-efficiency or complexity claim about its mechanism, whether it reports any measured temporal quantity attributable to that mechanism, and the figure, table or sentence that settled the decision. A statement such as "training took a few days" does not qualify, since it is not attributable to the mechanism. Scope matters for at least one entry, whose affirmative verdict rests on a wall-clock figure in an appendix and would be negative on the main body alone. This is an audit of reporting practice in a small, deliberately selected set — the papers we ourselves relied on — and not a sample of either literature; a negative verdict means the argument rests on operation counts or asymptotics, not that it is wrong.

[pending]

E Reproduction and provenance

The measurement campaign is training-free and completes in under an hour on one device. `scripts/run_revision_campaign.sh` regenerates every artifact in the order the manuscript reports them; `scripts/make_revision_tables.py` and `scripts/make_revision_figures.py` regenerate every table and figure from those artifacts, so no tabular number in the manuscript is transcribed by hand; and `scripts/check_paper_numbers.py` re-derives the numbers that appear in prose and fails on any disagreement.

Three provenance caveats. The hardware description and the aggregate provenance record were written before this project's environment was upgraded and describe a superseded software stack and a host that is no longer available; both have been corrected, and the run-level provenance for the training variants lives in the per-run configuration files rather than in the aggregate record. The GPUs used for every measurement in this paper are shared with other tenants, so absolute latencies are not reproducible to better than the drift reported in Appendix A.3 and only ratios between implementations timed in the same interleaved sweep should be compared. And the activation-memory measurement is reported from a run with the card otherwise free, since a shared-host repeat exhausts memory one agent count earlier.